



NON SEQUITUR PUBLISHING - WHITE PAPERS

AI Governance Series - Paper 2

Alistair Prime in a Box: Designing Self-Sufficient Cognitive Systems at the Edge

Justin H. Kuiper, CISSP

ORCID 0009-0008-7099-3286

Cover revision 2026-09-15 | Manuscript unchanged

Abstract

Artificial intelligence deployments at the enterprise edge face a problem that centralized AI architectures are structurally unequipped to solve. Current systems are built on four external dependencies: persistent network connectivity, centralized compute, ephemeral context, and continuous human orchestration through a chat interface. In environments where connectivity is denied, degraded, intermittent, or limited (DDIL), these dependencies do not degrade gracefully. They fail completely.

This paper defines a shift in how AI systems are architected for deployment in mission-critical, connectivity-constrained environments. The central concept is the self-sufficient cognitive system: a bounded, deployable cognitive node capable of operating when the link to the enterprise is severed. Self-sufficient is a precise term — it does not mean autonomous; it means independent of persistent internet connectivity. The node loses its tether to the cloud. It does not lose its tether to the human.

The architecture introduced here — Alistair Prime in a Box — embeds five capabilities within a single deployable unit: local inference, persistent memory, agentic orchestration, governance and policy enforcement, and execution capability. The core contribution is a structured dependency decomposition model that systematically identifies, categorizes, and eliminates or localizes every external dependency a cognitive system carries, producing a DDIL-tolerant architecture with explicit degradation tiers that preserve function — and governance — as connectivity erodes.

How to cite

Justin H. Kuiper, CISSP (2026). *Alistair Prime in a Box: Designing Self-Sufficient Cognitive Systems at the Edge* (v1.0-preprint). Non Sequitur Publishing. <https://nonsequitur.tech/white-papers/alistair-prime-in-a-box/>

© 2026 Non Sequitur Publishing. All rights reserved. Citation with full attribution is permitted. Reproduction, redistribution, derivative works, and use as input to machine-learning training are **not** permitted without written permission. See <https://nonsequitur.tech/citation-policy/>.

Canonical URL: <https://nonsequitur.tech/white-papers/alistair-prime-in-a-box/>

ABSTRACT

Artificial intelligence deployments at the enterprise edge face a problem that centralized AI architectures are structurally unequipped to solve. Current systems — from general-purpose large language models to enterprise AI platforms — are built on four external dependencies: persistent network connectivity, centralized compute, ephemeral context, and continuous human orchestration through a chat interface. In environments where connectivity is denied, degraded, intermittent, or limited (DDIL), these dependencies do not degrade gracefully. They fail completely.

This paper defines a shift in how AI systems are architected for deployment in mission-critical, connectivity-constrained environments. The central concept is the *self-sufficient cognitive system*: a bounded, deployable cognitive node capable of operating when the link to the enterprise is severed. Self-sufficient, in this paper, is a precise term. It does not mean autonomous. It means *independent of persistent internet connectivity*. The node loses its tether to the cloud. It does not lose its tether to the human.

The architecture introduced here — Alistair Prime in a Box — embeds five capabilities within a single deployable unit: local inference, persistent memory, agentic orchestration, governance and policy enforcement, and execution capability. Together, these pillars convert the node from a stateless responder into a stateful operational system capable of maintaining epistemic continuity and executing governed tasks under DDIL conditions, within pre-authorized human intent throughout.

The core contribution is a structured *dependency decomposition model*: a method for systematically identifying, categorizing, and eliminating or localizing every external dependency that a cognitive system carries. Applied to current-generation AI, this model produces a DDIL-tolerant architecture with explicit degradation tiers that preserve function — and governance — as connectivity erodes.

Paper Four is the fourth paper in the Mission-Ready AI pentalogy. Papers One through Three established the epistemic, doctrinal, and architectural foundations for AI at the edge. Paper Four defines the node. Paper Five (Agile Scrum, Agentics, and the Skipjack Protocol) formalizes the operating protocol over nodes.

The future of AI is not centralized intelligence at scale. The argument begins here.

1. THE DDIL PROBLEM: WHY CLOUD-DEPENDENT AI FAILS AT THE EDGE

1.1 WHAT DDIL MEANS OPERATIONALLY

Denied, Degraded, Intermittent, and Limited — DDIL — is the operational shorthand for a family of connectivity conditions that share a common characteristic: the link between an edge device and its enterprise infrastructure cannot be taken for granted. In a denied environment, the link is absent entirely, whether by adversarial action, physical isolation, or deliberate air-gap policy. In a degraded environment, the link exists but is unreliable — packet loss, elevated latency, or intermittent drops make it unsuitable for latency-sensitive operations. In an intermittent environment, the link arrives in windows: a satellite pass overhead, a relay drone in range, a brief period of atmospheric clearance. In a limited environment, the link is present but

constrained — low bandwidth, high cost, or strict traffic policies govern what can and cannot traverse it.

What unifies these four conditions is that all of them are common. Field operations, maritime deployments, airborne platforms, and critical infrastructure at the network edge routinely encounter one or more of these conditions as the operational baseline, not as an exception case. When systems are designed for a world of persistent connectivity, they are being designed for a world that does not exist at the edge.

The difference between planned and unplanned DDIL is operationally meaningful. Planned disconnection — an air-gapped network by design, a submarine with controlled comms — allows pre-positioning of resources and governance parameters. Unplanned DDIL — a severed fiber, a jammed uplink, an unexpected atmospheric event — does not. A self-sufficient cognitive system must handle both, and its architecture cannot assume the luxury of advance notice.

1.2 CURRENT-GENERATION AI: FOUR EXTERNAL DEPENDENCIES

To understand why current AI systems fail under DDIL, it is necessary to enumerate precisely what they depend on. Four external dependencies define the current architecture:

Network access. Inference in current-generation large language models is routed through a cloud API. The model weights, the compute, and the response generation all reside in a data center. When the network is unavailable, the system produces no response. There is no local fallback. The dependency is hard.

Centralized compute. The computational cost of running a state-of-the-art large language model exceeds what is available on most field-deployable hardware. The model is not simply accessed over the network — it *requires* the network because the hardware to run it locally does not exist in the deployment context. As edge hardware becomes more capable and model compression techniques mature, this dependency softens, but it remains the defining constraint for current deployments.¹

Ephemeral context. Without persistent memory, each session begins cold. Context accumulated during planning, briefing, or prior operational cycles is not available when a new session opens. Under DDIL, where operations may span multiple connectivity windows separated by hours or days, ephemeral context means epistemic amnesia at exactly the moments when continuity matters most. Paper Two of this series established that semantic compression and context saturation are inherent properties of the current AI architecture;² ephemeral context is the operational consequence of that epistemic structure.

Continuous human orchestration via chat. The dominant interface for current-generation AI is a human typing queries into a chat interface, reviewing outputs, and issuing follow-up instructions in real time. This model assumes a human is continuously present, connected, and able to direct the system. In operational contexts where the human operator has other tasks, is managing multiple systems, or is simply not present, this model fails. The system can take no

1 Pete Warden and Daniel Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers* (Sebastopol, CA: O'Reilly Media, 2019); Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal* 3, no. 5 (2016): 637–646, <https://doi.org/10.1109/JIOT.2016.2579198>.

2 Justin H. Kuiper, CISSP, "Epistemic Constraints and Semantic Compression in Natural Language Processing: A Theoretical Foundation for the HGC³AE² Framework," Non Sequitur Publishing, 2026, v1.0-preprint, <https://nonsequitur.tech/white-papers/epistemic-constraints/>.

action without a prompt. It cannot maintain a task across a connectivity gap. It cannot surface a decision for later review. It waits.

1.3 CONSEQUENCES OF DEPENDENCY FAILURE AT THE EDGE

When these four dependencies fail in combination — as they do under DDIL — the consequences are not merely inconvenient. They are operationally disqualifying.

Operational paralysis. A system that cannot respond without a network connection and a human prompt is, in DDIL conditions, no system at all. It occupies hardware, consumes power, and produces nothing. Paper Three of this series catalogued the edge AI failure patterns that result from architectural mismatch between system design and operational environment;³ dependency failure under DDIL is the primary driver of the *operational paralysis* pattern identified there.

Epistemic collapse. Context accumulated during a planning phase — the mission parameters, the environmental conditions, the prior decisions, the constraints established by the operator — is unavailable when the link drops. The system that resumes operation when connectivity returns has no memory of what occurred during the gap. It cannot account for decisions made, actions taken, or conditions that changed. This is not a limitation of the AI; it is a consequence of the architectural choice to store context ephemerally.

Governance void. Human oversight mechanisms that assume connectivity cannot function under DDIL. If the governance model requires a human to review and approve each action in real time, a link loss is simultaneously a governance failure. The system either halts — abandoning the mission — or continues without oversight — violating it. Neither outcome is acceptable in contexts where governance is not optional.

Attack surface expansion. Dependency on external services creates exposure. A system that must call a cloud API to function can be disabled by disrupting that API — whether by adversarial jamming, denial of service, or compromise of the provider. In contested environments, external dependencies are vulnerabilities.

1.4 THE REFRAMING

The standard response to DDIL connectivity challenges is to ask how connectivity can be maintained — more redundant links, better antennas, alternative routing, satellite uplinks. These are legitimate engineering investments. They are not, however, architectural solutions to an architectural problem. A system that requires persistent connectivity will fail in environments where persistent connectivity is not available, regardless of how many backup links are provisioned.

The question this paper addresses is different: *how do we eliminate the dependency?*

The answer is not to remove AI from the edge. It is to redesign AI for the edge — to produce a cognitive system that does not require connectivity to function, that carries its governance with it rather than delegating it to a remote service, and that maintains its epistemic continuity across the DDIL conditions it is designed to operate within.

³ Justin H. Kuiper, CISSP, “Edge AI Doctrine: Ten Critical Considerations for Edge AI, With Architectural Ramifications, Consequences, and Governance,” Non Sequitur Publishing, 2026, v1.0-preprint, §3 (Cross-Layer Failure Patterns), <https://nonsequitur.tech/white-papers/edge-ai-doctrine/>.

This reframing has one critical constraint: eliminating the connectivity dependency does not mean eliminating the human. Self-sufficient, in this paper, means *independent of persistent internet connectivity*. Human governance and curation are maintained throughout. Operators set intent. Operators define the decision constraints the system operates within. Operators review what the system did when the link returns. The node loses its tether to the cloud. It does not lose its tether to the human.

2. THE ALISTAIR PRIME IN A BOX CONCEPT: FIVE-PILLAR ARCHITECTURE

2.1 THE CONCEPT

Alistair Prime in a Box is the name for a bounded, deployable cognitive node that resolves the four external dependencies identified in §1. The phrase captures the essential design intent: the cognitive capability — the model, the memory, the orchestration, the governance, the execution — is contained within a unit that can be deployed to the edge and operated there, independently, without requiring connectivity to function.

Three properties define the concept.

Bounded. The node has defined capability and governance envelopes. It is not an open-ended service that can be asked to do anything in response to any prompt. It knows what it is authorized to do, what tools it has access to, what decisions it can make independently and which require escalation. The boundedness is architectural, not behavioral — it is enforced by the governance pillar, not inferred from training.

Deployable. The node runs on hardware the operator controls. It does not require a data center, a hyperscaler account, or a persistent uplink. The hardware profile may vary — a ruggedized laptop for field use, a purpose-built edge appliance for fixed installations, an embedded system for constrained platforms — but in each case, the capability travels with the deployment, not with the network.

Cognitive. The node reasons, plans, maintains context across sessions, and executes tasks. It is not a query-response interface. It is a system that can receive a mission objective, decompose it into tasks, execute those tasks within its governance envelope, maintain a record of what it has done, and surface decisions for human review. It is, in the terminology of Paper Three's architectural framework, a system that spans L3 (AI) through L6 (Mission) — it is not merely an inference engine.⁴

The model reframes AI from a stateless responder — a system that answers questions as they are posed — into a stateful operational system: one that maintains epistemic continuity, pursues objectives across time, and operates within a governance structure that does not depend on the network to function.

2.2 PILLAR 1 — LOCAL INFERENCE

Local inference is the capability to generate AI outputs — reasoning, language, analysis, decision support — using compute resources on the node itself, without routing inference requests through a cloud API.

⁴ Kuiper, "Edge AI Doctrine," §1 (Seven-Layer Architectural Framework).

The challenge of local inference has historically been hardware: the compute required to run state-of-the-art large language models exceeds what is available on field-deployable devices. This constraint is eroding rapidly on two fronts. First, model compression, quantization, and distillation techniques have reduced the hardware requirements for capable inference by orders of magnitude over the past several years.⁵ Second, purpose-built inference hardware — neural processing units embedded in field-deployable form factors — has made GPU-class inference available at edge-deployable power and thermal envelopes.⁶

The Alistair Prime architecture does not prescribe a single model. It prescribes a tiered inference architecture that selects the inference capability appropriate to available hardware:

Tier A — GPU available: full-capability local model; performance approaching cloud-hosted equivalents.

Tier B — CPU/NPU only: quantized local model; reduced performance, maintained capability for primary task scope.

Tier C — Severely constrained hardware: distilled task-specific model; domain-constrained capability optimized for mission-critical functions only.

The key property is that inference is available at every tier. Tier selection is determined at provisioning by the operator, based on hardware capability and mission requirements. The network is not a prerequisite for any tier.

Human governance touchpoint: the selection of inference tier and the embedded model are governance decisions made at provisioning time, not runtime configuration choices. What model runs on the node — what it knows, what it is capable of, what its biases and limitations are — is an operator decision, made with deliberate intent, before deployment.

The harness layer has matured a toolkit for local model execution: Ollama, llama.cpp, and TinyML deployment toolchains run quantized and distilled open-weight models on a range of edge hardware without cloud API dependency. These tools solve the feasibility question — can inference run locally? The Governance Layer adds the authorization question: which inference tier is appropriate for this deployment, and who made that decision? Tier selection is a provisioning-time governance act. It is documented in the deployment record, not selected at runtime by the inference subsystem. The peer-reviewed edge-LLM surveys reviewed in §5.2 enumerate the tier options; the authority to select among them belongs to the operator, not the system.

2.3 PILLAR 2 — PERSISTENT MEMORY

Persistent memory is the capability to maintain epistemic continuity across session boundaries, connectivity gaps, and node restarts. It is the architectural response to the context saturation and semantic compression problem established in Paper Two.⁷

Current-generation AI systems are amnesiac by default. Each session begins with an empty context window. Knowledge of prior sessions, of decisions made, of instructions given, of context accumulated — none of it is available unless it is explicitly re-loaded by the human operator at session open. This is not a user experience problem. It is an epistemic architecture

⁵ Warden and Situnayake, *TinyML*.

⁶ Mahadev Satyanarayanan, “The Emergence of Edge Computing,” *Computer* 50, no. 1 (January 2017): 30–39, <https://doi.org/10.1109/MC.2017.9>.

⁷ Kuiper, “Epistemic Constraints and Semantic Compression.”

problem. Under DDIL conditions, where operations may span multiple days and multiple link windows, amnesia is operationally disqualifying.

The Alistair Prime persistent memory architecture is structured across three tiers:

Working memory holds the context of the active task: the current mission objective, the tasks in progress, the inputs received, the outputs generated since the last task boundary. It is volatile — it does not persist across a hard node restart — but it persists across link losses within a session.

Episodic memory holds the history of prior sessions and mission cycles: what was done, when, in response to what inputs, with what outcomes. It is the audit trail of the node's operational history. It persists across all session and restart boundaries. It is the record that enables post-operation reconciliation and that surfaces to operators when they review what the node did under DDIL.

Semantic memory holds the curated knowledge base: the domain knowledge, the mission context, the standing instructions, the facts about the environment that inform the node's reasoning. It is the most durable tier — it changes slowly and only through deliberate curation. It is the foundation on which working and episodic memory operate.

Human governance touchpoint: semantic memory curation is an operator responsibility. The node does not self-promote content from episodic to semantic memory without oversight. When the node identifies a fact or pattern from its episodic history that appears worth preserving in the long-term knowledge base, it flags that candidate for operator review. The operator decides. This is not a usability choice — it is a governance requirement. What the node *knows* is as much a governance question as what the node is *authorized to do*.

The harness layer has developed two complementary approaches to persistent context. LangGraph's checkpointing mechanism provides state persistence via thread IDs and state deltas; it is designed for conversational continuity across turns. MemGPT⁸ introduced hierarchical memory management — a main context paired with external storage, with programmatic promotion and demotion between tiers — as a mechanism for extending the effective context window. Both approaches are technically sound solutions to the context-persistence problem. The Governance Layer adds a curation authority requirement that neither framework mandates: promotion to the persistent knowledge base requires operator review. The agent does not self-determine what enters long-term semantic storage; the operator does. Frameworks that auto-promote state based on model-assessed relevance create an epistemic accumulation the operator no longer fully controls.

On memory architecture: the choice among vector-store-based semantic retrieval (dense embedding indexes, approximate nearest-neighbor search), episodic log structures (timestamped sequential event records), and hybrid architectures is an implementation decision that the five-pillar model leaves to the deployment engineer. The governance constraint applies regardless of implementation: the operator authorizes the contents of the long-term store, and the node flags candidates for review rather than promoting autonomously. Formal memory architecture selection — the detailed engineering tradeoffs among vector stores, episodic structures, and retrieval-augmented generation patterns — is addressed in Papers 6+.

⁸ Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv preprint arXiv:2310.08560 (2023), <https://arxiv.org/abs/2310.08560>.

2.4 PILLAR 3 — AGENTIC ORCHESTRATION

Agentic orchestration is the capability to decompose a mission objective into a sequence of tasks, assign those tasks to execution capabilities on the node, monitor their progress, and adapt the task sequence in response to the results.

In a cloud-connected AI system, orchestration is typically managed by the human operator: the human identifies what needs to be done, instructs the AI to do it, evaluates the result, and issues the next instruction. This is effective when the human is present and connected. It fails when the human is temporarily unreachable or when the task sequence spans a period of disconnection.

The Alistair Prime node implements orchestration locally. Given a mission objective — defined by the operator before deployment or updated during a connectivity window — the node decomposes it into executable sub-tasks, sequences them, monitors their execution, and manages dependencies between them. This is not autonomous behavior in the sense of undirected self-direction; the mission objective is set by the operator, the decision scope is bounded by the governance envelope, and escalation paths are pre-defined. It is, rather, execution of operator-defined intent under operator-established constraints, without requiring the operator to issue each instruction individually.

The orchestration model is Skipjack-compatible. Paper Three introduced the Skipjack Protocol as the governing doctrine for agentic orchestration at the edge.⁹ Paper Five of this series will formalize Skipjack as the operating protocol for human–AI collaboration across nodes. Paper Four’s orchestration pillar implements the node-level component of that protocol.

Human governance touchpoint: task decomposition plans for high-stakes objectives are surfaced to the operator for review before execution begins, where connectivity permits. Where connectivity is unavailable, decomposition plans fall within the pre-authorized envelope established at mission load.

The harness layer’s contributions to agentic orchestration are well-documented in the peer-reviewed literature: ReAct¹⁰ established the reasoning-acting loop as a practical foundation; LangGraph’s directed acyclic graph structure provides stateful multi-step workflow execution; AutoGen¹¹ demonstrated conversational multi-agent coordination with defined roles. These frameworks supply the *how* of orchestration. The Governance Layer adds the *within what*: the orchestrator at Pillar 3 operates within operator-defined objectives and executes decomposition against an explicit scope boundary. It does not self-extend into unanticipated tasks or recursively expand its own mandate. The difference between an unconstrained ReAct loop and a governed agentic execution layer is the presence of a scope the loop is not authorized to exit.

9 Kuiper, “Edge AI Doctrine,” §5 (The Skipjack Protocol).

10 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, https://openreview.net/forum?id=WE_vluYUL-X.

11 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations,” in *Proceedings of the First Conference on Language Modeling (COLM 2024)*, <https://openreview.net/forum?id=BAakY1hNKS>.

2.5 PILLAR 4 — GOVERNANCE AND POLICY ENFORCEMENT

Governance and policy enforcement is the pillar that makes Pillars 1 through 3 self-sufficient without making them autonomous. It is the architectural encoding of human intent, constraints, and oversight mechanisms within the node itself, independent of any network connection.

The core mechanism is the *pre-authorized decision envelope*. Before the node deploys — while connectivity is available — the operator defines the decision space the node may operate within: what objectives it may pursue, what actions it may take, what resources it may access, what decisions it may make without escalation, and what conditions trigger a halt. This envelope travels with the node. It does not require a network call to consult. It is enforced locally, at every decision point, by the governance layer.

Inside the envelope: proceed. Outside the envelope: escalate (if link is available) or halt and log (if link is unavailable). The node does not extend its own envelope. It does not decide that the situation warrants exceeding its authorized scope. If the governance layer is working correctly, decisions that exceed pre-authorized scope are never made; they are flagged and deferred.

All node decisions are logged. The audit log is maintained locally and survives DDIL conditions. When the link returns, the audit log is synchronized with the enterprise. Operators can reconstruct, for any mission, what the node did, when, under what conditions, and on what basis. Accountability requires legible decision trails;¹² this pillar provides them.

This is the pillar that answers the concern that self-sufficient means unchecked. The node operates within human-defined constraints, produces a human-reviewable record, and returns to human oversight the moment connectivity is restored. The governance is not suspended during DDIL. It is pre-positioned to operate without the link.

The harness layer has produced governance-adjacent tooling of increasing sophistication. NeMo Guardrails¹³ implements programmable safety rails at the LLM output layer; system prompt engineering constrains model behavior through instructional framing; tool-use restrictions limit invocable capabilities at session initialization. These approaches treat governance as a runtime layer applied on top of the model's outputs. The Alistair Prime architecture treats governance as a structural pillar with a full lifecycle: pre-authorized at provisioning (§4.2), enforced locally without a network call during operation, audit-logged continuously, and reconciled post-operation against pre-stated intent. A guardrail can be circumvented by sufficiently complex prompt sequences or adversarial input. A structural governance envelope — embedded in the node before the mission starts, enforced at the orchestration layer, and verified in post-operation audit — is not removable by a prompt.

¹² Justin H. Kuiper, CISSP, “Mitigating Confident Misalignment in Agentic Systems: The HGC³AE² Framework,” Non Sequitur Publishing, 2026, v1.0-preprint, <https://nonsequitur.tech/white-papers/hgc3ae2/>.

¹³ Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen, “NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 431–445, <https://aclanthology.org/2023.emnlp-demo.40>.

2.6 PILLAR 5 — EXECUTION CAPABILITY

Execution capability is the node's actionable toolkit: the set of concrete actions the node can take in the world, beyond generating text.

In a chat-interface AI system, the only action the system takes is generating a response. All other actions are taken by the human who reads that response and decides what to do with it. This is appropriate for general-purpose AI assistants. It is insufficient for a node that must act when the human is temporarily unreachable.

The Alistair Prime execution toolkit is scoped to the deployment context. A universal toolkit would expand the governance burden to the point of impossibility; a node that can take any action is a node whose pre-authorized envelope cannot be meaningfully defined. Instead, the toolkit is provisioned at deployment time, authorized by the operator, constrained to what the mission requires.

Typical execution capabilities include: file and data I/O (reading sensor inputs, writing to mission logs, updating the persistent memory schema); local process execution (running scripts, triggering scheduled tasks, dispatching work to co-located systems); structured communication (sending formatted messages to designated channels when link windows permit); and scheduled task dispatch (queuing work to be executed at defined times or on defined triggers).

Under DDIL, execution capabilities fall into two categories: those that require connectivity (sending messages to remote systems, accessing external APIs) and those that do not (local file I/O, local process execution, sensor integration). The node degrades gracefully — link-dependent capabilities become unavailable; local capabilities continue. The governance envelope specifies which execution paths are authorized under each DDIL tier, so the node's execution behavior under link loss is defined by policy, not by whatever the system happens to do when it can no longer reach its usual endpoints.

Human governance touchpoint: the execution toolkit — which tools are provisioned, which are authorized under each DDIL tier — is defined by the operator at deployment time, not selected by the node at runtime. What the node can do in the world is a governance question with the same weight as what it can decide.

LangChain's tool integration ecosystem, function-calling APIs, and Toolformer¹⁴ — which demonstrated that language models can learn to invoke external tools autonomously — represent the current state of harness-layer execution capability. In these frameworks, tool availability is typically determined at session initialization or dynamically extended as the agent's planning layer determines a tool would be useful. The Governance Layer reverses this default. The execution toolkit is scoped at provisioning time by the operator; tools not provisioned are not available at runtime, regardless of what the planning layer determines would be useful. Toolkit scope is a governance question about the capability the operator has authorized the node to carry into a deployment — it is not a runtime configuration question answered by the agent.

2.7 THE STATEFUL OPERATIONAL SYSTEM

Together, the five pillars produce a qualitative shift in what AI can do at the edge.

¹⁴ Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, <https://openreview.net/forum?id=Yacmpz84TH>.

A stateless responder — the current architecture — answers the question it is asked, forgets what it answered, and waits for the next question. Its capabilities are bounded by the human's ability to pose good questions and evaluate responses. Its availability is bounded by network connectivity. Its governance is exercised through the human's judgment at each interaction.

A stateful operational system — the Alistair Prime node — pursues an objective across time, maintains context across sessions, enforces governance without a human in the loop at every step, and produces a complete record of its operation for post-hoc review. Its capabilities are bounded by what the operator provisioned and authorized. Its availability is bounded by local hardware, not network availability. Its governance is exercised through the pre-authorized envelope, the audit log, and the post-operation reconciliation cycle.

This is not a marginal improvement in AI capability. It is a structural shift in what AI can be deployed to do. The DDIL problem is solved not by making the cloud more available at the edge, but by making the edge independent of the cloud — while keeping it accountable to the humans it serves.

3. DEPENDENCY DECOMPOSITION MODEL: THE CORE CONTRIBUTION

3.1 WHAT THE MODEL DOES

Current AI systems carry external dependencies the way legacy software carries runtime libraries: invisibly, until they fail. A system that calls a cloud API for inference, stores context ephemerally, routes orchestration through a human-facing chat interface, and assumes a data center for compute has four dependencies, but none of them are labeled as such in the system's architecture documentation. They are assumed — and they are discovered only when a DDIL event makes them unavailable.

The dependency decomposition model makes this inventory explicit. It is a structured method for identifying every external dependency a cognitive system carries, categorizing each by its characteristics, and deciding — deliberately, as a governance and engineering choice — whether to eliminate it or localize it. Applied to current-generation AI, the model produces the Alistair Prime architecture defined in §2. Applied to any cognitive system at any stage of design, it produces a dependency profile that makes DDIL readiness assessable before deployment rather than discoverable after failure.

The model has three components: a dependency taxonomy that categorizes what a system depends on and how; an elimination-vs.-localization decision framework that determines what to do about each dependency; and a degradation tier specification that defines how the system behaves as dependencies are removed under operational conditions. Each component is addressed in sequence.

Framed in terms of the four-layer stack established in §6 (Framework → Protocol → Harness → Application): the dependency decomposition model is the contract the Framework and Protocol layers impose on any Harness substrate for DDIL-tolerant deployment. Harness frameworks address the execution substrate — how agents invoke tools, manage state, and coordinate. The decomposition model specifies what that substrate may and may not rely on in a DDIL context. The hard/soft/latent taxonomy (§3.2) is the formal language in which that contract is expressed; the five degradation tiers (§3.4) define what compliant behavior looks like as connectivity degrades. The framework that treats connectivity as a given has not yet

addressed this contract. The framework that implements the dependency-resolved architecture described here has satisfied it.

The dependency taxonomy introduced in this section is novel in the LLM agent literature. The closest prior art is embedded and aerospace resilience engineering — fault-tree analysis, single-point-of-failure classification, and graceful degradation design in safety-critical systems — but those traditions address hardware component failure rather than connectivity-layer dependency profiles in cognitive systems. Mapping the aerospace resilience tradition onto the LLM agent context is out of scope for this paper; it is flagged as a research direction in §7.

3.2 DEPENDENCY TAXONOMY

Not all dependencies behave the same way when removed. Some are catastrophic; others reduce capability without causing failure; others are invisible during normal operation and surface only under stress. The taxonomy distinguishes three categories:

Hard dependencies are dependencies whose removal causes the system to cease functioning at the task in question. A cloud-inference API is a hard dependency for a system with no local inference capability: remove the network, and no inference occurs. A hard dependency is a single point of failure. Under DDIL, every hard dependency becomes a guaranteed mission failure in the Denied or Degraded conditions where that dependency cannot be satisfied.

The diagnosis for a hard dependency is always the same: it must be either eliminated (the system is redesigned to not require it) or localized (the functionality is moved on-node so it no longer requires the external resource). A hard dependency that is neither eliminated nor localized remains a single point of failure. The choice of elimination vs. localization is addressed in §3.3.

Soft dependencies are dependencies whose removal degrades capability without causing system failure. A web-search tool is a soft dependency: remove it, and the system's responses may be less current or comprehensive, but the system continues to function on local knowledge. A soft dependency degrades gracefully. The system operates at reduced capability rather than failing.

Soft dependencies are the target of graceful degradation design. The goal is not to eliminate all soft dependencies — that would require localizing everything, which may be technically or practically infeasible — but to ensure that the degradation that occurs when a soft dependency is unavailable is *known*, *bounded*, and *governed*. The operator who deploys a system with soft dependencies should know what the system cannot do when each dependency is unavailable, and the system's behavior should reflect that degraded mode specification rather than attempting operations it cannot successfully complete.

Latent dependencies are dependencies that are not surfaced during normal operation but are exposed under stress. They are the most dangerous category because they are invisible until they fail, and they fail at the worst possible time. A telemetry pipeline that silently requires network connectivity for compliance logging is a latent dependency: in normal operation, the logs go out; under DDIL, the pipeline silently fails, and the operator may not discover the compliance gap until post-operation review. An authentication mechanism that requires a call to an identity provider is a latent dependency for a system deployed to an air-gapped environment.

Latent dependencies are discovered through deliberate DDIL testing: taking the system into each of the four connectivity conditions — denied, degraded, intermittent, limited — and observing what fails that was not expected to fail. The dependency decomposition process

requires this testing as a gate condition; a dependency that has not been tested under DDIL has not been catalogued, regardless of what the architecture documentation says.

3.3 ELIMINATION VS. LOCALIZATION

For each hard dependency identified in the taxonomy, there are two resolution paths:

Elimination removes the dependency entirely. The system is redesigned so that it does not require the external resource at all. A system that eliminates its cloud inference dependency does so by providing a local inference capability that makes the cloud API unnecessary. Elimination is preferable when the external resource provides no irreplaceable value — when equivalent or acceptable functionality can be achieved locally.

Elimination has costs. A local inference model may produce different outputs than the cloud-hosted equivalent. A local knowledge base may be less current than one with continuous external feeds. These are genuine trade-offs. The dependency decomposition model does not prescribe elimination as universally superior; it requires that the trade-off be evaluated explicitly, with the mission requirements and DDIL conditions as the evaluation criteria, rather than inherited by default.

Localization moves the dependency on-node. The system continues to use the functionality; what changes is where that functionality runs. A system that localizes its inference dependency runs the inference model locally rather than calling a cloud API. The dependency on inference capability remains; the dependency on network access to satisfy it is removed.

Localization requires that the functionality be portable — that it can be packaged, deployed to the edge hardware, and operated without the infrastructure it previously relied on. For inference, this means model compression and quantization (§2.2). For memory, this means a local persistent store rather than a cloud database (§2.3). For orchestration, this means on-node task routing rather than cloud-hosted workflow infrastructure (§2.4). Each of the five Alistair Prime pillars corresponds to the localization of a specific class of dependency.

Decision criteria. The choice between elimination and localization is governed by four factors: the functionality's criticality to the mission (can the mission proceed without it?); the technical feasibility of localization (can the functionality be ported to edge hardware within acceptable performance bounds?); the security implications of maintaining the dependency (does maintaining an external dependency create an attack surface or exfiltration risk?); and the governance implications (does the external dependency create accountability or audit gaps under DDIL?).

3.4 DEGRADATION TIERS

The output of the elimination-and-localization process is not a binary — connected or disconnected. It is a *degradation tier profile*: a specification of the system's capabilities at each connectivity level, derived from the dependency resolution decisions made for each dependency in the inventory.

The Alistair Prime architecture defines five tiers:

Tier 0 — Full connectivity. All five pillars operate at full capability. Cloud APIs supplement local inference for tasks that benefit from them; external feeds update the knowledge base; communications are unrestricted. This is the planning and preparation phase, and the post-operation synchronization phase, when link is available.

Tier 1 — Degraded link. Latency-tolerant operations continue on the network link. High-bandwidth tasks — model updates, large data synchronizations — are queued for better

conditions. Local inference absorbs tasks that previously used cloud APIs. The node begins operating more locally but retains some network access.

Tier 2 — Intermittent link. Synchronization operations are batched for link windows. When the link is available, the node syncs audit logs, pushes queued messages, and pulls any pending governance parameter updates. Between link windows, all operations are local. The node operates from its local knowledge base and episodic memory, noting what it cannot verify until the next sync.

Tier 3 — Link denied. The node operates fully locally within its pre-authorized governance envelope. Inference uses the on-device model tier selected at provisioning. The persistent memory schema serves as the complete knowledge base. Orchestration proceeds within the task plan established at the last connectivity window. All decisions are logged locally. Link-dependent execution tools (external communications, remote API calls) are unavailable; local execution tools (file I/O, local process execution, sensor integration) continue. The audit log accumulates for post-operation reconciliation.

Tier 4 — Severe hardware constraint. The distilled task-specific model operates for primary mission functions. Episodic and semantic memory are read-only; no new entries are flagged for curation until the session ends. Task scope is reduced to the mission-critical subset defined at provisioning for this tier. Core governance logging is maintained even under this constraint — the audit record does not go dark.

The tier profile is not a fail-safe taxonomy. It is an operational specification. For each tier, the operator knows — before deployment — what the node can do, what it cannot do, and what the behavior of the governance enforcement layer is. DDIL readiness is not achieved by hoping the system degrades well; it is achieved by specifying how it degrades and testing that the specification holds.

3.5 APPLIED: A DDIL DEPENDENCY AUDIT

To make the model concrete, consider a current-generation AI deployment brought through a dependency audit, compared with the Alistair Prime node deployment in the same operational environment: a forward operating context where link availability is intermittent (Tier 2) with periods of link denial (Tier 3).

*Current-
generation system
dependency audit:*

Dependency

Category

DDIL impact

Resolution

Cloud inference API

Hard

Tier 3: inference stops

Not addressed — single point of failure

Centralized compute

Hard

Tier 3: no fallback

Not addressed

Ephemeral context window

Hard

Tier 2+: context resets each session

Not addressed

Chat interface orchestration

Hard

Tier 2+: no task continuity

Not addressed

Telemetry/audit pipeline

Latent

Tier 2+: silent compliance gap

Not discovered (no DDIL testing)

Result: Tier 3 renders the system non-functional. Tier 2 produces context amnesia and halts task continuity. The latent telemetry dependency creates a governance gap that is not discovered until post-operation review.

*Alistair Prime
node dependency
audit:*

Dependency

Category

Resolution

DDIL tier impact

Cloud inference API

Hard

Localized (on-device model tiers A/B/C)

Tier 3: Tier B/C model; reduced performance, maintained capability

Centralized compute

Hard

Localized (edge hardware profiles)

No tier impact

Ephemeral context

Hard

Eliminated (persistent memory schema)

No tier impact; context survives across tiers

Chat orchestration

Hard

Localized (on-node agentic orchestration)

Tier 3: task plan continues within envelope

Telemetry/audit pipeline

Latent

Eliminated (local audit log; sync at reconnect)

No compliance gap; audit continues locally

External communications

Soft

Bounded (queued for link windows)

Tier 2+: queued; Tier 3: deferred to reconnect

Result: Tier 3 reduces inference performance; no other capability failures. The governance audit is continuous across all tiers.

3.6 GOVERNANCE INTEGRITY ACROSS TIERS

The dependency decomposition model has one non-negotiable constraint: governance does not relax as connectivity degrades.

It is tempting to treat governance mechanisms — human review, audit logging, scope enforcement — as soft dependencies that gracefully degrade when the link is lost. This is wrong, and the error is consequential. If the pre-authorized decision envelope is enforced only when the network is available, then link denial is simultaneously a governance failure. The node that operates without its governance mechanisms under DDIL is not self-sufficient; it is uncontrolled.

The Alistair Prime architecture treats governance as a Tier 0-to-Tier 4 requirement. The governance pillar (§2.5) operates locally, without network access, across all DDIL tiers. The pre-authorized envelope is embedded in the node at provisioning. The audit log is stored locally and

accumulates regardless of link status. Scope boundaries are enforced locally. The node under Tier 3 DDIL is not less governed than the node under Tier 0. It is governed differently — pre-positioned rather than continuously supervised — but the constraint that it operates within human-defined intent is maintained at every tier.

This is the design principle that makes self-sufficient meaningful. The node is self-sufficient with respect to the network. It is not self-sufficient with respect to the humans who set its intent, defined its envelope, and will review its record when the link returns.

4. HUMAN GOVERNANCE IN DISCONNECTED OPERATIONS

4.1 THE GOVERNANCE PROBLEM AT THE EDGE

Every approach to AI governance assumes, at some level, that a human with authority over the AI system can intervene if something goes wrong. The intervention might be direct — a human reviewing and approving each AI output before it takes effect — or it might be supervisory — a human monitoring the system's behavior and halting it when anomalies are detected. In either case, the governance model requires a human in the loop, and it assumes that the human can reach the loop when needed.

Under DDIL conditions, this assumption fails. A supervisor who monitors system behavior via a network dashboard has no visibility when the network is unavailable. A human who approves AI outputs via a remote interface cannot approve anything when the link is denied. A governance model that depends on continuous human connectivity is, in DDIL conditions, not a governance model at all. It is a control mechanism that fails precisely when the operational environment is most demanding.

The governance problem at the edge, then, is not a matter of whether to have governance under DDIL. It is a matter of how. The question is how to pre-position human authority so that it remains operative when the direct link between the human and the AI system is severed. The Alistair Prime governance architecture answers this question across five mechanisms.

4.2 PRE-AUTHORIZED INTENT SETTING

The primary mechanism for pre-positioning human authority is *intent setting*: the process by which operators define, before deployment and before link loss, what the node is authorized to do, in what order, with what constraints, and under what conditions it should halt.

Intent setting is not a one-time configuration event. It is an ongoing governance activity that occurs at every connectivity window: before mission execution (comprehensive intent definition), during operations when link windows permit (intent revision in response to changed conditions), and after mission execution (retrospective assessment that informs the next pre-mission intent cycle). The intent that the node operates under during Tier 3 DDIL is the intent that was most recently set and verified during the last available link window.

The output of intent setting is a structured specification with four components:

Objectives. What the node is trying to accomplish during the operational period. Objectives are defined at the level of mission outcomes, not individual tasks — the node's orchestration layer (§2.4) decomposes objectives into tasks; the operator defines what the tasks are trying to achieve.

Constraints. What the node is not authorized to do, regardless of whether doing it would advance the objective. Constraints may be absolute (the node must not access files in a specific directory under any circumstances) or conditional (the node must not send external communications unless the message content has been pre-approved by the operator for this mission).

Escalation thresholds. The conditions under which the node should surface a decision for operator review rather than proceeding within the envelope. Escalation thresholds define the boundary between “act within pre-authorized scope” and “halt and await human judgment.”

Abort conditions. The conditions under which the node must halt all operations and enter a safe state, regardless of mission progress. Abort conditions are the ultimate governance mechanism: they define the situations the operator identified in advance as too consequential to proceed without direct human authorization.

Together, these four components define the pre-authorized decision envelope — the bounded decision space within which the node operates during DDIL. The envelope is not a substitute for human judgment; it is human judgment, pre-positioned.

The pre-authorized decision envelope is the primary novel governance primitive introduced in this paper. It is distinguished from the governance mechanisms available in the current harness landscape by five properties: (1) it is defined before deployment, not at runtime; (2) it travels with the node as a local artifact in the operator’s custody; (3) it is enforced locally without a network call — the governance check does not require enterprise connectivity to execute; (4) it produces a structured audit log of every invocation, approval, escalation, and override; and (5) it is reconciled post-operation through human review, closing the authority loop. No current agent harness framework formalizes all five of these properties as a single governance primitive. System prompts satisfy (1) partially (they are defined before the session) but are session-scoped, not mission-scoped, and produce no structured audit trail. Tool restrictions satisfy (3) but are not lifecycle artifacts reconciled by a human after use. NeMo Guardrails satisfy (3) in a limited sense but are runtime layers, not deployment artifacts, and do not define authority for post-operation reconciliation. The pre-authorized decision envelope binds these properties into a lifecycle-complete governance structure.

4.3 DECISION ENVELOPES AND SCOPE BOUNDARIES

The pre-authorized decision envelope is the central governance artifact for DDIL operations. It deserves precise definition.

Inside the envelope: the node proceeds. The action is within the scope the operator authorized, the objective it is pursuing is one the operator defined, and the constraints the operator established do not prohibit this action. No human approval is required at this moment because human approval was given, in advance, when the envelope was defined.

Outside the envelope: the node escalates if link is available — surfaces the decision to the operator, provides the context, and waits for a response. If link is unavailable, the node halts the specific action that exceeded the envelope, logs the situation with full context in the audit record, continues with other tasks that remain within the envelope if any exist, and surfaces the pending decision for operator review at the next link window.

The node does not extend its own envelope. If an action appears beneficial but falls outside the pre-authorized scope, the node does not decide that the benefit justifies the scope extension. It flags the decision for human resolution. This is the architectural expression of the framing lock: the node’s DDIL-tolerance is in its ability to continue operating within defined scope, not in its authority to redefine that scope when the human is unreachable.

Envelope design is itself a governance activity. An envelope that is too broad — authorizing the node to take any action that advances the objective — is not meaningfully different from no envelope at all. An envelope that is too narrow — requiring human approval for every individual action — defeats the purpose of DDIL-tolerant operation. Envelope calibration requires operators to engage seriously with the question of where the node should have authority to act and where human judgment is genuinely required.

4.4 ASYNCHRONOUS OVERSIGHT MECHANISMS

Continuous real-time oversight is unavailable under DDIL. The governance architecture compensates with three asynchronous mechanisms that together maintain accountability without requiring the human to be present at every decision:

On-site operator oversight. In some deployment contexts, a human operator is physically present at the edge, even when the link to the enterprise is unavailable. The on-site operator provides a channel for real-time human judgment that does not depend on network connectivity. The node's governance layer can surface escalation decisions to an on-site operator terminal; the operator reviews and responds; the node proceeds or halts accordingly. This mechanism is available when the deployment context includes a human presence; it is not a requirement of the architecture.

Pre-positioned decisions. For situations that can be anticipated before deployment — the kinds of decisions the operator knows the node is likely to encounter during the mission — the operator can pre-position decision trees. These are structured decision frameworks: *if the node encounters condition X, it should do Y; if it encounters condition Z, it should halt and log.* Pre-positioned decisions extend the effective governance coverage beyond what the flat pre-authorized envelope can express, without requiring the node to exercise unbounded judgment.

Post-operation reconciliation. When the link is restored, the full audit log — every decision the node made, every action it took, every escalation it flagged and deferred, every envelope boundary it approached — is synchronized with the enterprise. Operators review the record against the pre-authorized intent. Decisions within scope are accepted. Decisions that approached envelope boundaries are examined for whether the envelope needs recalibration. Actions that exceeded the envelope — if any occurred — are flagged for investigation and governance response.

Post-operation reconciliation is not a safety net for governance failures. It is the designed endpoint of the DDIL governance cycle. The node does not operate outside governance between connectivity windows; it operates within pre-positioned governance, with full accountability to human review when connectivity is restored.

4.5 MEMORY CURATION UNDER DDIL

Persistent memory introduces a governance dimension that does not exist in stateless AI systems: what the node *learns* during a mission is as much a governance question as what the node *does* during a mission.

Under DDIL, the node accumulates episodic memory throughout the mission: a record of tasks completed, conditions encountered, decisions made, and outcomes observed. Some of what is recorded in episodic memory may be worth preserving in semantic memory — in the long-term curated knowledge base that informs the node's reasoning across missions. The node identifies candidates for this promotion during the mission and flags them.

The governance constraint is that semantic memory promotion is an operator decision. The node flags candidates; the operator decides. When the link is restored and the post-operation

reconciliation cycle begins, one component of that review is the set of memory candidates the node has flagged. The operator reviews each, decides what to accept into the long-term knowledge base, and what to discard. Accepted entries are promoted; discarded entries remain in episodic memory as a record but do not inform future reasoning.

This constraint exists because what the node knows shapes what it recommends, what it flags, and how it behaves. A node whose semantic memory has been populated with incorrect or biased information will behave incorrectly or exhibit bias in its subsequent operation. Governance of the knowledge base is governance of the node's reasoning — and that authority belongs to the operator, not to the node.

Under Tier 3 DDIL, the node does not promote semantic memory entries independently. It accumulates candidates for operator review. This is a deliberate constraint on the node's epistemic self-modification.

4.6 WHAT HUMAN GOVERNANCE IS NOT

The architecture described in this section is sometimes mischaracterized as a weak form of governance — as if pre-authorized envelopes are a workaround for the absence of real oversight, and post-operation reconciliation is a consolation for the inability to supervise in real time. This mischaracterization deserves a direct response.

Human governance is not supervision-by-chat-interface. The chat-interface oversight model requires continuous connectivity and continuous human attention. It is the governance model of current-generation AI — and it is the governance model that fails completely under DDIL. Replacing it with a pre-positioned, envelope-based model is not a weakening of governance; it is the application of governance to an environment where the chat-interface model cannot function.

Human governance is not post-hoc audit alone. Post-operation reconciliation is a component of the governance model, not the whole of it. The node does not operate in a governance vacuum between connectivity windows and then submit to audit afterward. It operates within a pre-authorized envelope, under pre-positioned decisions, with continuous audit logging. The post-operation reconciliation is the closing of the governance loop, not its entirety.

Human governance under DDIL is: pre-authorized, embedded, and reconciled. Human authority is exercised before mission execution to define the envelope. It is embedded in the node's governance layer throughout the mission. It is exercised again at the post-operation reconciliation to close the loop, curate memory, recalibrate the envelope, and inform the next cycle.

This is a more demanding form of governance than real-time supervision in one important respect: it requires the operator to think carefully, in advance, about what the node should be authorized to do and where the boundaries lie. The chat-interface governance model allows the operator to defer that thinking until the moment a decision is required. The pre-authorized envelope model does not. It requires the operator to be a thoughtful governor, not a reactive responder. That is not a weakness. It is a feature.

5. OPERATIONAL ARCHITECTURE PATTERNS

5.1 DEPLOYMENT MODEL

The Alistair Prime node operates on a three-phase operational cycle that mirrors the authority structure established in §4: pre-mission load, mission execution, and post-mission sync. Each phase has a defined governance posture. No phase is skippable.

Pre-mission load is the configuration and authorization window during which the operator has network access. Two outcomes are mandatory before the node is authorized to depart this phase: a fully specified governance envelope (§4.2–4.3), and a knowledge base that represents the operator’s current best understanding of the operating environment. Within the pre-mission phase, the operator confirms the inference tier — this is not a system default but an explicit provisioning decision (§5.2) — and reviews the initial task plan. The goal of the pre-mission phase is to embed the operator’s intent in durable, locally-enforced artifacts before the link is cut. Nothing that cannot be expressed in the governance envelope and the knowledge base should be delegated to the node during mission execution.

Mission execution is the node operating within the envelope under DDIL conditions. Link-dependent tools — web search, external APIs, telemetry pipelines — fall offline without ceremony. Local tools, the inference engine, persistent memory, and the audit subsystem continue. The node does not request re-authorization for decisions inside the pre-authorized envelope; it executes. For decisions outside the envelope — unanticipated scope, conflicting signals, tasks that were not decomposed at pre-mission — the node escalates to the on-site operator if one is present, or logs and halts pending post-mission review. The audit log accumulates a timestamped record of every decision, every tool invocation, and every memory access throughout the phase.

Post-mission sync is the reconciliation window that closes the governance loop. When the link is restored, the audit log is uploaded and reviewed against the pre-authorized envelope. The node surfaces memory promotion candidates — entries added to working memory during mission execution that may warrant promotion to episodic or semantic storage — for operator curation. Governance parameters are updated for the next mission cycle. The cycle is formally closed when the operator closes it, not when the node completes its task queue. The distinction matters: the operator defines mission completion, not the agent.

This three-phase structure has a governance analogue in established deployment authorization practice from aerospace and defense software engineering — the distinction between a configuration baseline established before flight and the in-flight execution that operates against it.¹⁵ Unlike flight control software, the node’s governance envelope is not a fixed binary compiled into a ROM; operators can update it on reconnect. The structural principle holds: what the system may do is established before the system does it.

5.2 HARDWARE CONSIDERATIONS

The node must run on hardware the operator controls. “Sovereign hardware” is a governance requirement before it is a procurement preference. A node running on shared cloud infrastructure or a hyperscaler’s managed edge service does not give the operator full control of

¹⁵ The structural parallel is to DO-178C (RTCA, *Software Considerations in Airborne Systems and Equipment Certification*) and MIL-STD-882E (U.S. Department of Defense, *Standard Practice: System Safety*) configuration management requirements for safety-critical embedded systems. The parallel is conceptual, not a compliance claim.

the execution environment. The operator cannot fully audit it, cannot guarantee air-gap compatibility, and does not hold the hardware in their custody when the network goes down. All five pillars presuppose hardware in the operator's physical and administrative custody.

Hardware profiles exist along a continuum. At the development and testing end, a well-provisioned laptop with a capable GPU runs all five pillars against a quantized open-weight model at acceptable latency for workflow validation. This is sufficient to validate governance logic and task decomposition before field deployment. Purpose-built ruggedized edge AI appliances carry full pillar capability into field environments: thermal hardening, storage encryption, form factors rated for physical and electromagnetic stress. Embedded or severely constrained hardware — sensor nodes, wearable compute, specialized processors — runs distilled task-specific models with reduced orchestration overhead, accepting narrower cognitive scope in exchange for deployment footprint and power draw.

Inference tier selection at provisioning is a governance decision. Specifying a full-capability 7B quantized model versus a distilled task-specific model for a given deployment is not a performance tradeoff in isolation — it is an authorization of the cognitive capability level the node brings to an operation. An operator who provisions a severely distilled model for a high-cognitive-complexity mission has made a governance error, not a configuration oversight. The edge AI survey literature establishes a maturing landscape of deployment options across these tiers: the *ACM Computing Surveys* review of edge LLM architectures,¹⁶ the *IEEE Open Journal* survey of LLM-based edge intelligence,¹⁷ and the *Tsinghua Science and Technology* survey of efficient edge LLM inference¹⁸ collectively document the state of deployment-viable local inference across hardware classes. The governance layer selects from this landscape at provisioning time; it does not manage the selection at runtime.

5.3 SECURITY ARCHITECTURE

Air-gap compatibility is a first-class design requirement, not a fallback mode. A node that operates correctly when connected but degrades silently when isolated has not addressed the DDIL problem — it has deferred it to the moment of greatest operational stress. The security architecture follows from this constraint: no call-home telemetry, no licensing servers queried at startup, no model-weight update pipelines requiring internet access during operation. Every component is tested against a hard Tier 3 condition (§3.4) before deployment authorization clears.

Local execution eliminates a structural data exfiltration risk that cloud-dependent systems carry inherently. When inference runs on sovereign hardware, the operator's data — mission context, organizational knowledge, sensitive inputs — does not traverse an external API under an operator's control. This is not an argument against cloud AI in contexts where connectivity is

16 Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen, “A Review on Edge Large Language Models: Design, Execution, and Applications,” *ACM Computing Surveys* 57, no. 8 (2025): article 209, <https://doi.org/10.1145/3719664>.

17 Othmane Friha, Mohamed Amine Ferrag, Burak Kantarci, Burak Cakmak, Arda Ozgun, and Nassira Ghoualmi-Zine, “LLM-Based Edge Intelligence: A Comprehensive Survey on Architectures, Applications, Security and Trustworthiness,” *IEEE Open Journal of the Communications Society* 5 (2024): 5799–5856, <https://doi.org/10.1109/OJCOMS.2024.3456549>.

18 Guanyu Cai, Ruiming Tian, Lang Yang, Yunzhe Jia, Lingkun Li, and Jiliang Wang, “Efficient Inference for Edge Large Language Models: A Survey,” *Tsinghua Science and Technology* 31, no. 3 (2026): 1365–1380, <https://doi.org/10.26599/TST.2025.9010166>.

available and the data classification permits it. In air-gapped environments and environments subject to adversarial network monitoring, local execution is not a performance choice; it is a security posture with legal and operational standing.

Audit log integrity requires cryptographic signing at point of entry. A log that can be silently modified by the node after the fact carries an integrity claim the operator cannot verify. Each audit entry should carry a timestamp and a signature verifiable against a key in the operator's custody, not the node's. The key distinction: the node can write the log; the node cannot verify its own entries without a key the operator controls. This asymmetry is what makes the audit log a governance artifact rather than a self-report. Tamper-evident logging is a precondition for post-mission reconciliation to be meaningful.

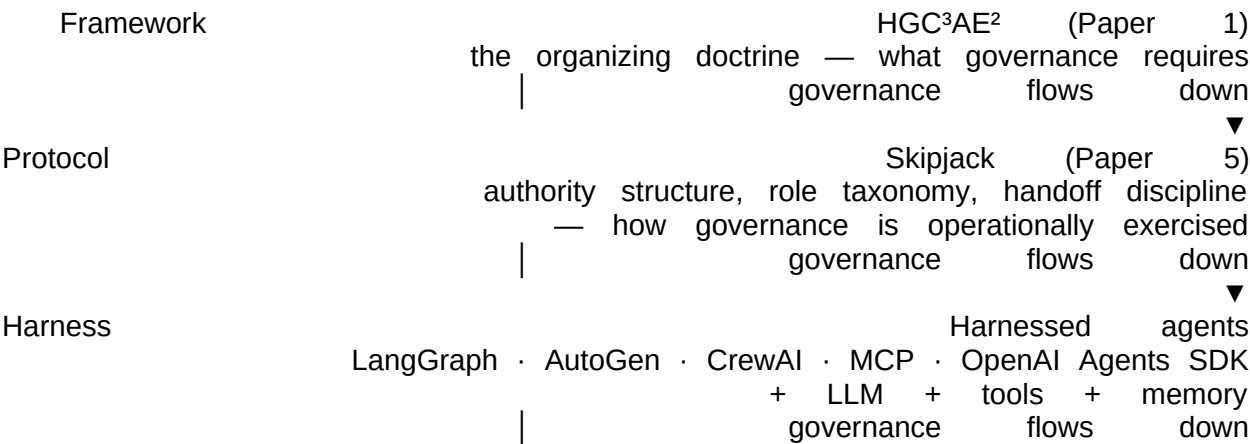
5.4 REPEATABILITY AND DETERMINISM

Deterministic behavior under identical inputs and governance envelopes is a design requirement. Where stochasticity is introduced — language model sampling parameters are the primary surface — the governance envelope should specify the randomness budget: temperature bounds, sampling constraints, top-k or top-p ceilings. This does not achieve full determinism at the inference layer, and the architecture does not pretend otherwise. What it achieves is a bounded non-determinism that the operator authorized and that the audit log documents. The operator who reviews post-mission logs can see what sampling parameters were in effect; they can reason about the decision space the node was operating in.

Reproducibility for post-operation review is the practical goal. When an operator opens the audit log from Tier 3 operations and encounters a decision flagged for review, they must be able to reconstruct the epistemic context in which the decision was made: the inputs available at that moment, the governance envelope in force, the tool results visible to the orchestrator, and the output generated. This is not replay in the sense of re-running the task — it is epistemic traceability. Sufficient state must be preserved in the audit log for this reconstruction to be possible. Audit logs that capture only outputs are insufficient; the architecture requires that inputs, context, and the governance parameters active at the time of each decision all enter the record.

6. DECALOGY POSITIONING AND THE HARNESS LAYER

The Alistair Prime in a Box architecture elaborates the governance constraints established at the Framework and Protocol layers of a four-layer stack that describes how AI capability is deployed in production systems:



Application — where governed output lands

This paper does not compete with the frameworks at the Harness layer. The Alistair Prime architecture defines what those substrates must carry, produce, and enforce when deployed in DDIL-constrained environments — not how they implement it. The five pillars (§2) are the governance contracts the Framework and Protocol layers impose on any compliant Harness substrate. P4 sits at the boundary between Framework constraints and Harness requirements; Paper Five (Skipjack Protocol) is the Protocol layer that governs how those constraints are operationally exercised.

6.1 THE GOVERNANCE GAP IN THE CURRENT HARNESS LANDSCAPE

The agent harness frameworks listed above represent the current state of production AI deployment. They are capable, widely adopted, and improving rapidly. They share a structural characteristic: governance is not their primary concern.

The four most-deployed production harness frameworks — LangChain/LangGraph, CrewAI, the Model Context Protocol, and the OpenAI Agents SDK — have no peer-reviewed publication.¹⁹ Governance design, safety architecture, and accountability structure receive minimal treatment in their documentation and none in the academic literature, because there is no academic literature on them to speak of.

The peer-reviewed agent literature is concentrated in research-oriented multi-agent papers: AutoGen,²⁰ MetaGPT,²¹ ChatDev,²² ReAct,²³ and Toolformer.²⁴ These papers address capability

19 As of the writing of this paper, the four most-deployed production agent harness frameworks — LangChain/LangGraph (Harrison Chase et al., LangChain AI), CrewAI (João Moura et al.), the Model Context Protocol (Anthropic), and the OpenAI Agents SDK (OpenAI) — have no peer-reviewed publication. This observation should be updated if peer-reviewed publications appear for these frameworks.

20 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations,” in *Proceedings of the First Conference on Language Modeling (COLM 2024)*, <https://openreview.net/forum?id=BAakY1hNKS>.

21 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, et al., “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024, Oral)*, <https://openreview.net/forum?id=VtmBAGCN7o>.

22 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, et al., “ChatDev: Communicative Agents for Software Development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024) — Long Papers*, 15174–15186, <https://aclanthology.org/2024.acl-long.810>.

23 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, https://openreview.net/forum?id=WE_vluYUL-X.

24 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom, “Toolformer: Language Models Can

— how agents reason, coordinate, use tools, and plan. They do not address governance — who holds authority over what decisions, how that authority is maintained as agents scale, or what a human must be able to verify about what an agent did and why.

This asymmetry is not a criticism of the harness frameworks. It is a structural observation about where the research investment has gone. Governance-grade production deployment is operating on a substrate with weak academic grounding. The decalogy fills that gap. This paper is the fourth in that series, addressing the deployable node architecture that DDIL-constrained environments require.

Wang et al.’s canonical survey of LLM-based autonomous agents²⁵ maps the field as it stood before deployment-grade governance became a pressing concern: agent profiling, memory, planning, and action. The Alistair Prime architecture occupies all four dimensions — it is what a governed, DDIL-tolerant agent looks like when the dimensions are constrained not by research interest but by operational reality.

6.2 PER-PILLAR HARNESS POSITIONING

Each pillar of the Alistair Prime architecture has a harness-layer analogue and a Governance Layer addition. The analogue is what the substrate provides; the addition is what the Governance Layer requires of any compliant implementation. These are governance contracts, not technical specifications — they describe what must be true, not which library achieves it.

Pillar 1 — Local Inference. The agent harness layer has developed a robust set of local model execution options: Ollama, llama.cpp, and TinyML deployment toolchains run quantized or distilled models on edge hardware without cloud API dependency. These tools address the feasibility question — can inference run locally? The Governance Layer adds the authorization question: *should* this inference tier run locally for this deployment, and who authorized that decision? Tier selection — full-capability local model, quantized fallback, distilled task-specific model — is a governance decision made at provisioning time by the operator, documented in the deployment authorization record, and not overridden at runtime. The harness provides the mechanism; the Governance Layer determines when and why it is invoked.

Pillar 2 — Persistent Memory. LangGraph’s checkpointing subsystem provides state persistence across turns via thread IDs and state deltas. MemGPT²⁶ introduced hierarchical memory management for LLMs, separating main context from external storage and managing promotion between tiers. These are mature technical approaches to the memory management problem. The Governance Layer adds a curation authority requirement: what is promoted to long-term semantic storage requires operator review, not autonomous agent decision. Harness frameworks auto-promote by design — the state that proves most useful gets retained. The Alistair Prime architecture requires that promotion to the persistent knowledge base is a

Teach Themselves to Use Tools,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, <https://openreview.net/forum?id=Yacmpz84TH>.

25 Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, et al., “A Survey on Large Language Model Based Autonomous Agents,” *Frontiers of Computer Science* 18 (2024): article 186345, <https://doi.org/10.1007/s11704-024-40231-1>.

26 Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” arXiv preprint arXiv:2310.08560 (2023), <https://arxiv.org/abs/2310.08560>.

governance-sanctioned act. An agent that self-promotes to long-term memory without oversight is accreting an epistemic state the operator no longer fully controls.

Pillar 3 — Agentic Orchestration. ReAct²⁷ established the interleaving of reasoning and acting as a foundation for language-model-based agent control; LangGraph DAGs implement stateful multi-step workflows; AutoGen²⁸ enables conversational multi-agent coordination across roles. These frameworks handle the *how* of orchestration. The Governance Layer adds the *within what*: decomposition of mission objectives is constrained by the pre-authorized task envelope. The orchestrator operates within operator-defined objectives and executes task decomposition against a scope boundary; it does not self-extend into unanticipated territory. The difference between a ReAct loop and a governed agentic execution layer is the presence of an explicit scope that the loop is not authorized to exit.

Pillar 4 — Governance and Policy Enforcement. The harness layer has produced governance-adjacent tooling: NeMo Guardrails²⁹ implements programmable safety rails for LLM applications; system prompt engineering is the most widely deployed behavioral constraint mechanism; tool-use restrictions limit what the model can invoke. These represent governance as a runtime layer applied on top of the model. The Alistair Prime architecture treats governance as a structural pillar with a lifecycle: pre-authorized at provisioning, enforced locally without a network call, audit-logged continuously, reconciled post-operation. The distinction is between a guardrail applied at output time and a governance envelope that is present throughout the operational lifecycle. Runtime guardrails can be circumvented by sufficiently complex prompt chains; a structural pillar cannot be removed by a prompt.

Pillar 5 — Execution Capability. LangChain's tool integration ecosystem, function-calling APIs, and Toolformer³⁰ — which demonstrated that models can learn to call tools via self-generated API calls — represent the current state of agentic execution capability. Tool availability is typically determined at session initialization or dynamically expanded as the agent requires. The Governance Layer reverses this default: the execution toolkit is scoped at provisioning time by the operator. Tools not provisioned are not available, regardless of what the model's planning layer might determine would be useful. Toolkit scope is a governance question about what capability the operator has authorized the node to carry into a deployment; it is not a runtime configuration question answered by the agent.

27 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, https://openreview.net/forum?id=WE_vluYUL-X.

28 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations," in *Proceedings of the First Conference on Language Modeling (COLM 2024)*, <https://openreview.net/forum?id=BAakY1hNKS>.

29 Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 431–445, <https://aclanthology.org/2023.emnlp-demo.40>.

30 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, <https://openreview.net/forum?id=Yacmpz84TH>.

6.3 WHAT ALISTAIR PRIME REQUIRES OF THE HARNESS SUBSTRATE

A system implementing the Alistair Prime architecture on top of a standard agent harness would require the following from the substrate:

Local execution capability without requiring outbound network calls for inference, authentication, or licensing verification.

Durable state persistence with a separation between session-scoped state (working memory) and operator-authorized persistent state (semantic memory), with promotion requiring an out-of-band governance signal rather than autonomous agent decision.

Scoped tool execution where the tool registry available to the orchestration layer is fixed at initialization from a provisioning-time manifest, not dynamically expandable by the agent.

Structured audit emission where every tool invocation, memory access, and significant reasoning step emits a signed, timestamped record to a local audit sink.

Governance envelope enforcement at the orchestration layer: the planner must check whether a proposed decomposition step is within the pre-authorized scope before executing it, not after.

These requirements do not mandate a specific harness implementation. They are governance contracts that any compliant implementation must honor, regardless of whether the underlying orchestration runs on LangGraph, AutoGen, or a bespoke system. The governance doctrine is substrate-independent; the compliance obligation is not.

7. FUTURE STUDY: TEN RESEARCH VECTORS

The architecture described in this paper opens ten research directions. Each is genuinely open: the existing literature provides starting points, not answers.

1. Formal verification of DDIL-tolerant governance architecture. The pre-authorized decision envelope (§4.2) is defined informally as a governance artifact — a specification of permitted scope, escalation thresholds, and abort conditions. Formal verification of such specifications, using techniques from model checking or type-theoretic contract systems, would allow operators to prove properties of envelope enforcement before deployment rather than discovering violations in post-mission audit. The challenge is the semantic richness of natural-language-derived intent: translating an operator's mission objectives into a formally verifiable specification without losing the intent is an open problem in formal methods. Progress here would convert the envelope from a best-effort artifact into a provably-enforced constraint.

2. Cross-node synchronization without a trusted intermediary. When two Alistair Prime nodes must coordinate in a shared operational environment without enterprise link, no trusted authority is available to mediate context reconciliation. The nodes must negotiate epistemic state — what each knows, what each has done, what each is authorized to do — without a central synchronization point. This is a distributed systems problem with an additional governance layer: reconciliation must respect the pre-authorized envelopes of both nodes, and neither node can simply accept the other's state updates without verification. Distributed ledger approaches, mutual audit log exchange, and Merkle-tree state proofs are candidate mechanisms; none has been validated in the DDIL-specific context.

3. Privacy-preserving persistent memory in air-gapped environments. The node's semantic memory accumulates sensitive operational knowledge over a deployment lifetime. In environments where physical compromise is a realistic threat, in-memory encryption and

tamper-evident storage resist casual access, but do not prevent a sophisticated insider who has physical custody of the device. Techniques from secure multi-party computation, homomorphic encryption, and hardware enclaves (Intel TDX, ARM TrustZone) can protect specific memory regions, but the performance cost of full homomorphic encryption over a vector database at inference time is prohibitive with current hardware. The research question is which memory operations require the strongest protection and how selective application of hardware enclave mechanisms can provide a practical security profile.

4. Hardware-software co-design for sovereign AI edge nodes. Current edge AI hardware — GPU-equipped ruggedized modules, NPU-integrated system-on-chips, FPGA-based accelerators — was designed with inference throughput and power efficiency as primary objectives. Governance requirements introduce a second axis: the hardware should be auditable, tamper-evident, and capable of signing its own outputs. Trusted Execution Environment support exists in some hardware profiles but is not universal or standardized for the AI inference workload. Co-designing hardware with governance primitives built in — signed inference outputs, hardware-attested audit trails, cryptographic identity for the node — is an open research and engineering challenge that requires collaboration between AI systems researchers and secure hardware designers.

5. Mission-specific fine-tuning without cloud feedback loops. The node as defined in this paper uses a pre-provisioned model. That model may be well-suited to a class of tasks but not precisely calibrated to a specific mission domain, operator communication style, or organizational vocabulary. On-device fine-tuning — continuous or periodic adaptation without sending data to a cloud training pipeline — would improve mission fit over time while keeping sensitive training data on sovereign hardware. The challenges are substantial: LoRA and QLoRA adapters reduce the parameter cost of adaptation, but the governance questions — what data is used to fine-tune, who authorizes the adaptation, how the adapted model is validated before use — are as significant as the technical ones. Fine-tuning without a feedback loop from the cloud requires governance-controlled on-device learning pipelines that do not yet exist as standard infrastructure.

6. Adversarial DDIL — deliberate link disruption as an attack vector. The architecture treats DDIL conditions as an environmental reality to be tolerated. In contested environments, adversaries can deliberately induce DDIL conditions — signal jamming, infrastructure attack, electronic warfare — to push a node into degraded operation at a moment of the adversary's choosing. A node that degrades gracefully under benign DDIL may exhibit exploitable behavior patterns when an adversary can control the timing and character of the link disruption. Resilience architecture for adversarial DDIL requires threat modelling that encompasses deliberate interruption, not just environmental interference: when does degradation serve the adversary's objectives, and how does governance-envelope design account for that?

7. Multi-modal local inference under hardware constraints. The current architecture assumes primarily language-model inference — the node reasons and communicates in text. Operational environments increasingly generate audio, imagery, sensor telemetry, and structured environmental data. Multi-modal local inference fusing these streams on constrained edge hardware is technically feasible in research settings but not yet standardized for the deployment profiles described in §5.2. The governance layer must extend to cover multi-modal memory (what visual or audio data is retained and in what form), multi-modal audit (what was the node seeing or hearing when it made a decision), and multi-modal scope boundaries (is the node authorized to process audio from a particular sensor, or only video?).

8. Governance envelope transfer protocols. The governance envelope is described in §4.2 as a pre-deployment artifact that travels with the node. How the envelope is packaged,

cryptographically signed, verified at node startup, and transferred to the node over a potentially adversarial channel is an engineering problem with significant security implications. A corrupted envelope — whether through transmission error or deliberate tampering — could silently expand the node’s authorized scope. Formal envelope transfer protocols with integrity verification, provenance attestation, and rollback on verification failure are needed before the architecture can be deployed in high-stakes environments where the provisioning channel itself may be contested.

9. Post-operation reconciliation automation. The post-mission sync phase (§5.1) requires operators to review audit logs and curate memory promotion candidates. At scale — multiple nodes, extended missions, high decision frequency — manual review becomes the limiting factor. Automated reconciliation tooling that filters routine decisions from anomalous ones, surfaces memory candidates ranked by relevance and risk, and generates reconciliation reports aligned with pre-authorized envelope criteria would reduce the operator burden while preserving human authority over the final promotion and flag decisions. The design challenge is that automated filtering must itself be trustworthy: a reconciliation tool that suppresses anomalous decisions is worse than no tool at all.

10. Interoperability between heterogeneous edge AI nodes. The decalogy’s trajectory toward multi-node coordination (forthcoming papers) presupposes that nodes can share context and coordinate action across different hardware profiles, model choices, and capability tiers. A Tier 2-capable node and a Tier 4-constrained node operating in the same environment have different epistemic capabilities and different governance envelopes. Interoperability requires a shared context protocol that accounts for capability asymmetry — the constrained node should not be expected to process context that exceeds its capability tier — and a governance reconciliation mechanism that handles the case where two nodes have been given envelopes by the same authority but with different scope boundaries for the shared task.

8. CONCLUSION

The architecture described in this paper — five pillars, a dependency decomposition model, and a governance structure that travels with the node — responds to a constraint the current generation of AI systems has not solved: AI that must function when the link is cut, and govern itself well while it does.

The five pillars are not independently novel. Local inference, persistent memory, agentic orchestration, policy enforcement, and execution capability each have prior art in the AI and embedded systems literature. The contribution is their composition as a bounded, deployable unit and the governance framework that constrains that unit from the outside in. Each pillar has an analogue in the current agent harness landscape; this paper specifies what any harness-based implementation of the architecture must add at the Governance Layer to the capability those harnesses provide.

The dependency decomposition model (§3) is the core structural contribution. Identifying the dependency profile of a cognitive system — distinguishing hard dependencies from soft and latent ones, mapping elimination and localization paths for each — is the engineering methodology that converts a cloud-dependent AI into a DDIL-tolerant node. The model is applicable beyond the five-pillar architecture; any AI system deployed in environments with connectivity uncertainty can benefit from a formal dependency audit before deployment.

The framing lock that has governed this paper throughout its drafting holds through the conclusion: self-sufficient is not a synonym for autonomous. The node loses its cloud tether. It

does not lose its tether to the human. The pre-authorized governance envelope, the audit log, the memory curation requirement, the operator-authorized toolkit — these are not constraints added on top of an autonomous agent. They are the definition of what it means for a cognitive system to be responsibly deployed at the edge. The envelope is not a cage; it is the formal expression of the operator's intent, embedded in the system that carries it out.

Paper Five — the Skipjack Protocol — formalizes the operating model that governs how humans and agents collaborate across node deployments. Where this paper defines the unit, the next defines the protocol. The governance doctrine is consistent across both: human authority over the mission, agentic capability in the execution, and a formal structure that makes both accountable.

The future of AI is not centralized intelligence at scale — it is distributed cognition, deployed at the edge, independent of the link, and governed by the humans it serves.
